
CONCEPTION D'UN PROCESSEUR : LE DÉCODEUR

1 Introduction : le décodeur

Dans les séances précédentes, vous avez conçu une **ALU** (Unité Arithmétique et Logique) ainsi qu'un banc de registres. Ces composants matériels permettent d'effectuer des opérations de base (addition, soustraction, ET/OU logique, etc.) sur des données stockées dans les registres. Cependant, l'ALU seule ne sait pas quelle opération effectuer ni sur quels registres travailler.

Le **décodeur** est un bloc chargé de décoder l'instruction afin de paramétrer le reste du processeur pour l'exécution de cette instruction. Concrètement, par rapport à ce que nous avons déjà construit, **le décodeur nous permettra de sélectionner la bonne opération à effectuer dans l'ALU ainsi que de gérer les registres en lecture et en écriture en fonction de ce que l'instruction demande.**

Le décodeur est le « cerveau » qui :

- **Analyse l'instruction** en entrée, représentée par son opcode (ex : ADD, R1, R2, R3);
- **Décode l'opcode** pour déterminer quelle opération l'ALU doit effectuer (ex : addition);
- **Configure les bits de sélection** :
 - **Pour l'ALU** : active le signal correspondant à l'opération (ex : ADD, SUB, etc.);
 - **Pour les registres** : sélectionne les registres sources (Rs1, Rs2) et le registre destination (Rd) via des bits de contrôle;
- **Transmet les données** entre les registres et l'ALU via le bus de données.

Objectif de ce TP : Concevoir une **partie du décodeur** qui, à partir d'une instruction binaire en entrée, ajuste les signaux de contrôle de l'ALU et du banc de registres afin d'exécuter l'instruction.

Dans cette partie, nous chercherons uniquement à exécuter une seule instruction qui sera fournie en entrée du décodeur. **Il n'y aura pas de RAM pour l'instant**, ni d'instructions liées à la RAM telles que LOAD ou STORE. En conséquence, il n'y aura pas non plus de gestion du pointeur d'instruction (IP), ni de boucle, ni de saut conditionnel. Nous nous concentrerons simplement sur le **paramétrage de l'ALU et du banc de registres**. Les registres pourront par ailleurs être pré-remplis.

2 Implémentation du décodeur

Caractéristiques propres à notre processeur CYT-VX8

Afin de simplifier la mise en place du décodeur et du reste du processeur, nous avons défini certaines spécificités pour le processeur :

- Toutes les instructions seront codées sur **24 bits**;
- Comme indiqué dans les TP précédents, le résultat d'une comparaison devra toujours être placé dans un registre spécial 1 bit (qui est normalement déjà dans votre banc de registres) que nous appellerons le registre **RCMP** (pour *register comparison*). Ce registre servira donc de **flag Z**, tel que vu dans le cours;
- L'opcode est imposé (voir plus loin).

Ressources et outils

Pour cette étape et les suivantes, nous mettons à votre disposition sur la page du cours de Moodle :

- Un programme Python et son fichier **JSON** qui génère l'opcode d'une opération donnée en assembleur CYT-VX8;
- Un important *user guide* du processeur. Ce manuel contient l'architecture générale du processeur ainsi que l'intégralité des instructions qu'il peut exécuter et les opcodes associés. **Seules les instructions mentionnées dans les TP sont obligatoires**, mais vous pouvez implémenter d'autres instructions données dans ce manuel si vous le souhaitez.

Implémentation du décodeur

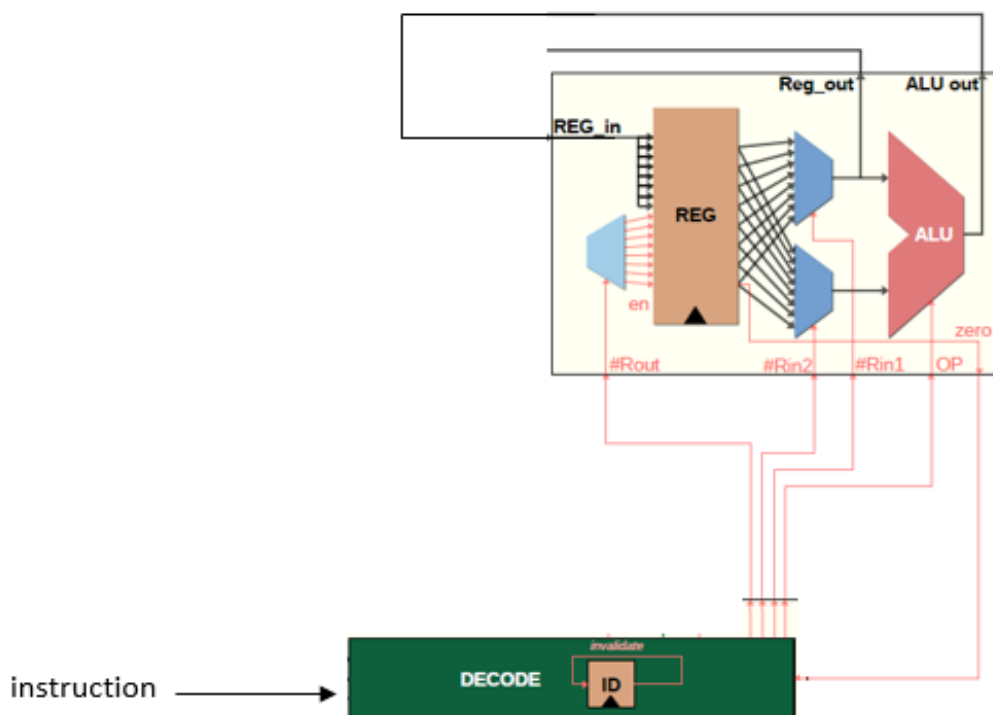


FIGURE 1 – Lien entre le décodeur et l'ALU

Afin de garantir la synchronisation de tout le processeur, le décodeur contiendra un registre de 24 bits permettant de stocker la prochaine instruction à décoder.

Il aura donc :

- en entrée : l'instruction à réaliser dans un cycle d'horloge ;
- en sortie : les différents paramètres pour l'ALU et les registres.

Le décodeur sera donc composé de :

- un registre de 24 bits ;
- en sortie de ce registre, un circuit logique combinatoire pour décoder l'instruction et obtenir les bonnes sorties.

À la fin de ce TP, votre processeur pourra donc déjà réaliser des opérations simples à partir d'une instruction.

Format des opcodes

Le format des instructions du CYT-VX8 est fixé à **24 bits**. Chaque instruction est structurée comme suit :

TABLE 1 – Format général d'une instruction (24 bits)

Type	Opération	Rin1	Rin2/Imm	0000	Rout
4 bits	4 bits	4 bits	4 bits	4 bits	4 bits

- **4 premiers bits** : Type d'opération (ex : 0001 pour les opérations arithmétiques à 2 registres).
- **4 bits suivants** : Opération spécifique (ex : 1000 pour ADD).
- **4 bits** : Registre source 1 (Rin1).
- **4 bits** : Registre source 2 (Rin2) ou valeur immédiate.
- **4 bits** : Champ inutilisé pour l'instant (mis à 0000).
- **4 derniers bits** : Registre destination (Rout).

La valeur du champ de registre équivaut au numéro de registre (ex 0010 pour R3, 0101 pour R6).

Pour les **instructions spéciales** (ex : JMP), les **bits 4 à 11** contiennent des adresses ou des constantes.

Opcode des instructions à implémenter

Ci-dessous se trouvent l'ensemble des détails des instructions à implémenter dans votre circuit :

L'instruction NOP

Cette instruction n'en est pas vraiment une : elle permet de faire passer un cycl d'horloge sans rien faire soit en ne modifiant pas les registres de données. L'ALU ne doit pas faire d'opération (ou bien ne doit pas stocker son résultat dans les registres).

Cette instruction sera nécessaire pour pouvoir par la suite effectuer les opérations de JUMP. Il est donc important de bien l'implémenter.

Son opcode est 0000 0000 0000 0000 0000 0000.

Instructions arithmétiques (2 registres)

TABLE 2 – Opcode des instructions arithmétiques à 2 registres

Mnémonique	Format	Opcode Binaire	Opcode Hex.	Description
ADD	ADD, Rin1, Rin2, Rout	0001 1000 Rin1 Rin2 0000 Rout	0x18XX0X	Addition Rout = Rin1 + Rin2
SUB	SUB, Rin1, Rin2, Rout	0001 1001 Rin1 Rin2 0000 Rout	0x19XX0X	Soustraction Rout = Rin1 - Rin2

Instructions logiques (2 registres)

TABLE 3 – Opcode des instructions logiques à 2 registres

Mnémonique	Format	Opcode Binaire	Opcode Hex.	Description
AND	AND, Rin1, Rin2, Rout	0001 0000 Rin1 Rin2 0000 Rout	0x10XX0X	ET logique bit à bit
OR	OR, Rin1, Rin2, Rout	0001 0001 Rin1 Rin2 0000 Rout	0x11XX0X	OU logique bit à bit
XOR	XOR, Rin1, Rin2, Rout	0001 0010 Rin1 Rin2 0000 Rout	0x12XX0X	OU exclusif bit à bit

Instructions de comparaison

Pour ces instructions, le registre de destination est toujours RCMP.

TABLE 4 – Opcode des instructions de comparaison

Mnémonique	Format	Opcode Binaire	Opcode Hex.	Description
CMP_EQ	CMP_EQ, Rin1, Rin2	0000 0011 Rin1 Rin2 0000 0111	0x03XX07	RCMP = 1 si Rin1 == Rin2
CMP_NE	CMP_NE, Rin1, Rin2	0000 0110 Rin1 Rin2 0000 0111	0x06XX07	RCMP = 1 si Rin1 != Rin2
CMP_GT	CMP_GT, Rin1, Rin2	0000 0101 Rin1 Rin2 0000 0111	0x05XX07	RCMP = 1 si Rin1 > Rin2
CMP_LT	CMP_LT, Rin1, Rin2	0000 0001 Rin1 Rin2 0000 0111	0x01XX07	RCMP = 1 si Rin1 < Rin2

Instructions logiques (1 registre) (si vous avez fait le bonus)

Pour les instructions à un seul registre, Rin2 est mis à 0000.

TABLE 5 – Opcode des instructions logiques à 1 registre

Mnémonique	Format	Opcode Binaire	Opcode Hex.	Description
LSL	LSL, Rin1, Rout	0000 1001 Rin1 0000 0000 Rout	0x09X00X	Décalage logique à gauche de 1 bit